Система за менижиране на лекарски посещения - *Дамян Мяшков 12в 7№*

1. Архитектура на проекта

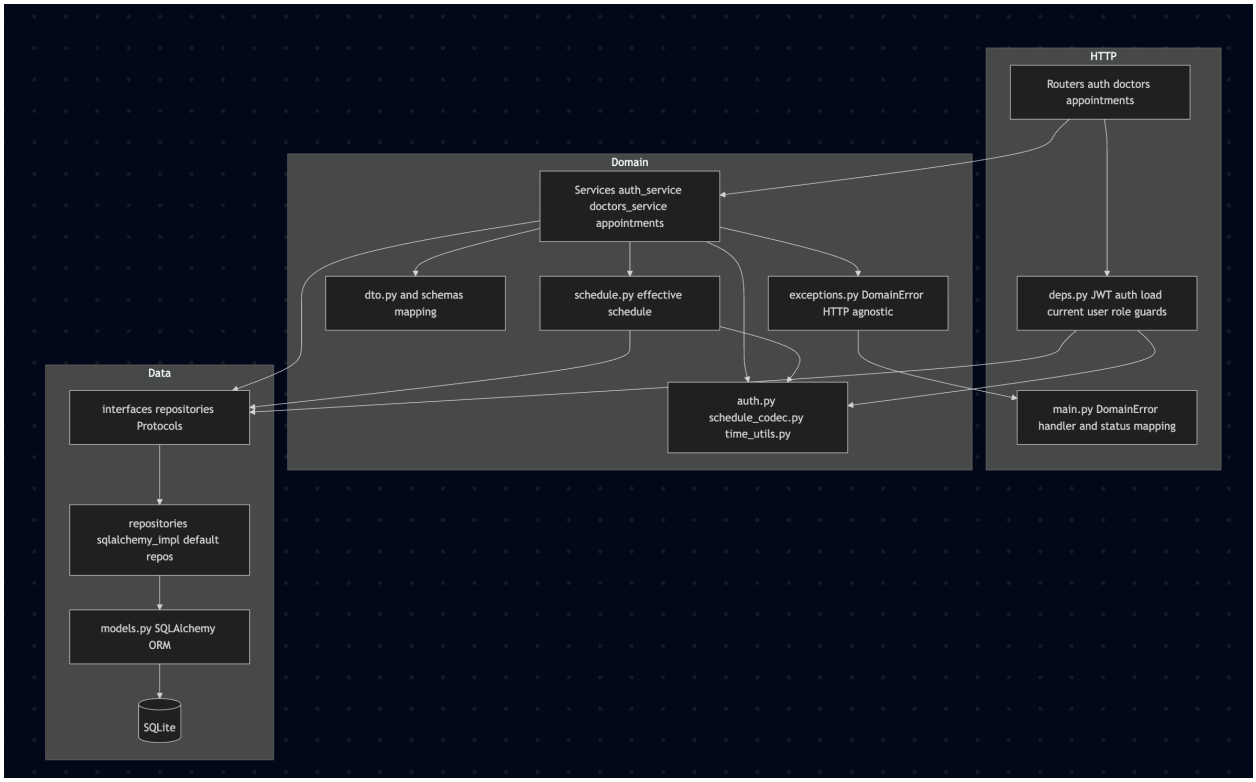
Слоеста структура отделя HTTP маршрутизацията, бизнес правилата, достъпа до базата и помощните домейн модули. По-долу са ключовите модули, структури и обосновка без излишно визуално форматиране.

1.1. Технологичен стек

- FastAPI - HTTP API, dependency injection, OpenAPI.
- SQLite - файлова база (app.db по подразбиране).
- SQLAlchemy 2 - ORM и сесии.
- Alembic - миграции.
- Pydantic v2 - валидация на вход/изход.
- python-jose - JWT; в sub се пази user id.
- pytest - тестове с in-memory SQLite във fixture.

1.2. Слоеве и отговорности

Архитектурата следва слоя Router → Service → Repository, като разделя отговорностите и прави кода по-четим и лесен за поддръжка. Router-ите дефинират HTTP пътищата, валидират входа и извикват съответните service-и. Service слоят съдържа основните правила и поведение на приложението, без да се занимава директно с базата данни. Repository слоят е единственото място, където се извършват заявки към базата чрез ORM и се управляват транзакции. Тази структура позволява ясно разграничение между API, приложната логика и достъпа до данни, като намалява дублирането и улеснява тестването и разширяването на системата.



Routers (app/routers/: auth, doctors, appointments) - пътища и методи; валидирани тела от Pydantic (app/schemas.py); авторизация чрез app/deps.py;

Services (app/services/: auth_service, appointments, doctors_service, schedule) - бизнес правила (24 h, 12 h, overlap, личен лекар, ефективен график).

- Връщат DTO от app/dto.py, не ORM.
- При нарушение: service функциите хвърлят DomainError (подкласове), които са HTTP-агностични. В HTTP слоя (app/main.py) има централен handler, който ги мапва към JSON {"detail": ...} и подходящ HTTP статус чрез отделен mapper

Repositories (app/repositories/) - единствено място за Session заявки и commit по таблици (users, doctor_profiles, patient_profiles, appointments, schedule_changes). Цел: тънък router/service без разпръснат SQLAlchemy.

Interfaces - Между services и конкретната SQLAlchemy имплементация има интерфейсен слой за repositories (Protocol), който позволява подмяна с fake имплементация при unit тестове или смяна на персистентността.

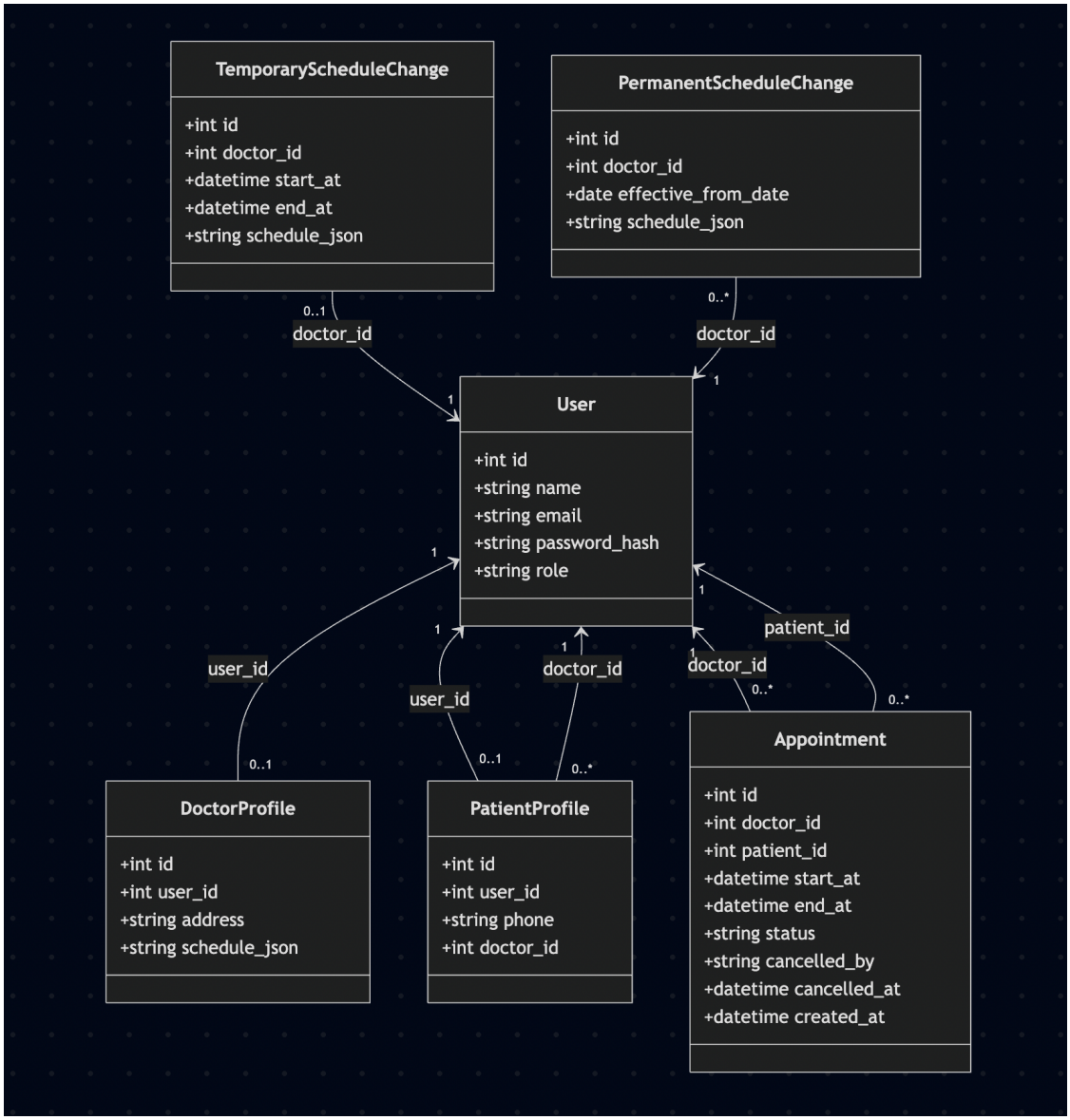
deps.py - get_current_user (JWT + зареждане на User и профил), require_doctor / require_patient за ограничаване на маршрути.

Други помощни модули:

- **auth**.py (PBKDF2, JWT),

- **schedule_codec**.py (график),
- **time_utils**.py (UTC),
- **enums**.py,
- **exceptions**.py,
- **settings.py** (BaseSettings)
- **models.py** - ORM класове;
- **db.py** – db helpers.
- **dto.py**: DTO се използва като междинен формат за данни между service слоя и API слоя. Service функциите не връщат директно SQLAlchemy ORM обекти, а връщат DTO структури (например DoctorDTO, PatientDTO, AppointmentDTO), които съдържат само нужните за отговора полета и нямат поведение, свързано с базата. Router-ите получават тези DTO резултати и ги мапват към Pydantic response модели, които сериализират JSON за клиента. Така API слойът остава независим от ORM детайли, избягват се проблеми с lazy loading и се гарантира, че към клиента излизат само контролирани и предвидими данни.
- interfaces/**repositories**.py - Protocol интерфейси за repository операции
- app/repositories/**sqlalchemy_impl**.py - default SQLAlchemy имплементация на интерфейсите
- **http_error_mapping**.py - мапване DomainError тип към HTTP status

1.3. Ключови ORM структури (полета и връзки)



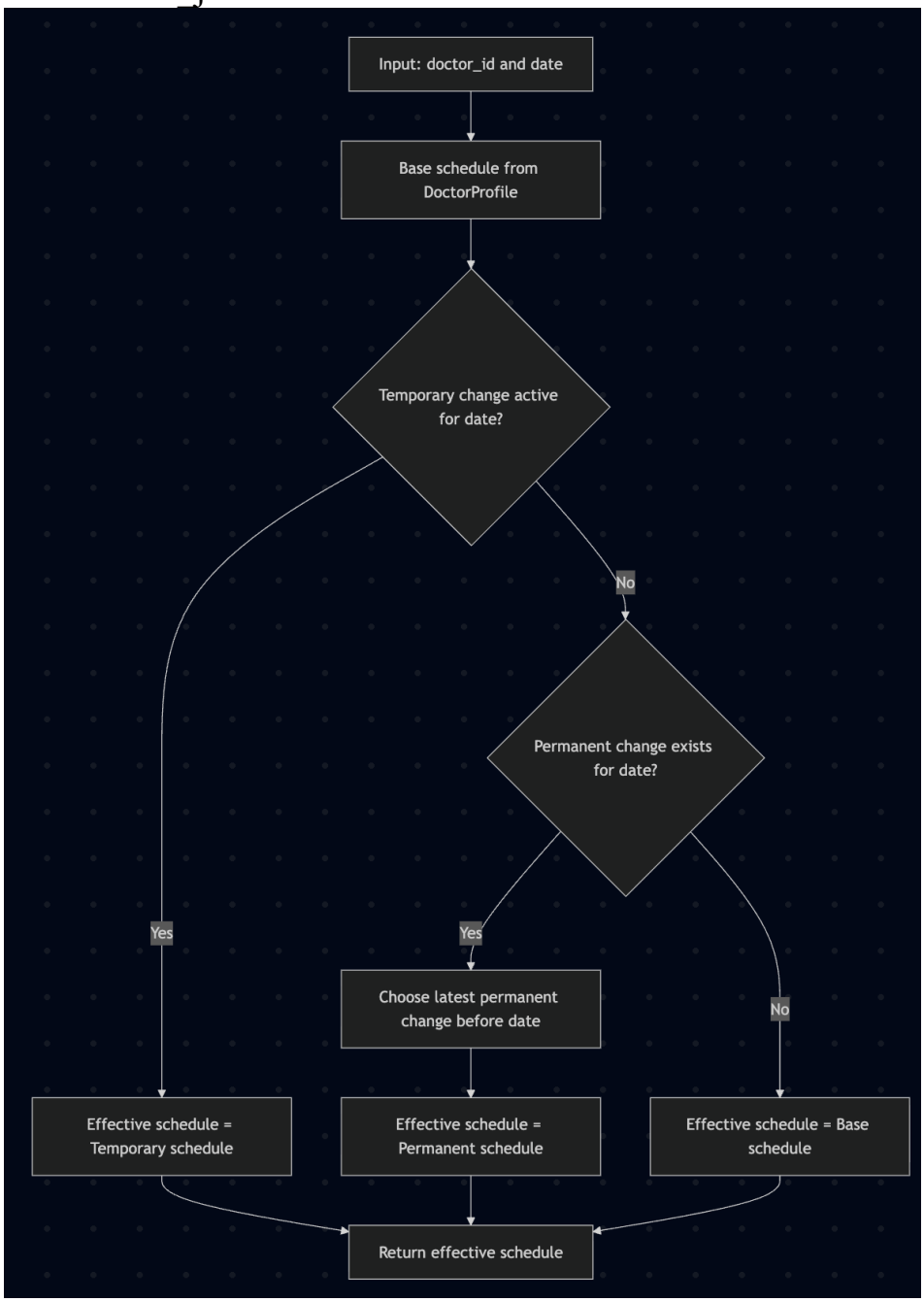
Моделът използва обща таблица User като основа за всички потребители, независимо от ролята им, където се съхраняват общите данни (име, имейл, парола, роля и метаданни). В зависимост от ролята, към всеки потребител е свързан допълнителен профил чрез 1:1 връзка — DoctorProfile за лекари и PatientProfile за пациенти. Така специфичната информация (като адрес и график за лекарите или телефон и личен лекар за пациентите) е отделена от базовите данни, което избягва излишни полета и поддържа структурата чиста и гъвкава.

- **User** - id, име, имейл (уникален), password_hash, role (enum), метаданни за създаване.
- **DoctorProfile** - 1:1 към User (user_id); address; schedule_json за базов график.

- **PatientProfile** - 1:1 към User; phone; doctor_id → users.id на личния лекар.
- **Appointment** - doctor_id, patient_id → users.id; start_at, end_at; status; cancelled_by, cancelled_at; created_at.

Ефективен график:

- **TemporaryScheduleChange** - най-много един ред на лекар (unique doctor_id).
- **PermanentScheduleChange** - много редове; effective_from_date; schedule_json.



1.4. Бизнес правила и място в кода

• **Създаване на посещение** (patient, POST /appointments)

  • Пациентът да има личен лекар и подаденият doctor_id да е точно този.

  • Началото да е поне 24 часа напред.

  • start_at и end_at да са в един и същи календарен ден.

  • Интервалът да попада в ефективното работно време на лекаря за тази дата.

  • Да няма overlap с друго активно посещение при същия лекар.

  • Място в кода - app/services/appointments.py, функция create_appointment; app/services/schedule.py, функция get_effective_schedule (работно време).

• **Отмяна на посещение** (doctor/patient, POST /appointments/{id}/cancel)

  • Посещението да съществува.

  • Текущият потребител да е лекарят или пациентът по това посещение.

  • Да остават поне 12 часа до началото на посещението.

  • При успех - статус cancelled, cancelled_by и cancelled_at.

  • Място в кода - app/services/appointments.py, функции cancel_appointment_as_user и cancel_appointment.

• **Списък „моите посещения“** (GET /appointments/me)

  • Ако потребителят е лекар - връща посещенията при него.

  • Ако е пациент - връща неговите посещения.

  • Подрежда по start_at.

  • Място в кода - app/services/appointments.py, функция list_my_appointments.

• **Ефективен график** (реално работно време за дата)

  • Ако има временна промяна, която покрива датата - тя замества базовия график.

  • Иначе се прилага последната постоянна промяна с effective_from_date <= датата; ако няма - остава базовият.

• Място в кода - app/services/schedule.py, функция get_effective_schedule.

• **Регистрация** (POST /auth/register-doctor, POST /auth/register-patient)

• Създава User и съответния профил в една транзакция.

• Валидира входа чрез Pydantic schema.

• При дублиран email - 409 (conflict).

• Място в кода - app/services/auth_service.py, функции register_doctor и register_patient.

1.5. Поток на грешките

- Невалиден вход от Pydantic: отговор 422 с detail списък.
- DomainError от service: централен handler в app/main.py връща JSON {"detail": ...} и HTTP статус чрез http_error_mapping.py.
- Липсващ/невалиден token в dependencies → 401/403 чрез HTTPException (Bearer).

2. Ползване на API

2.1. Автоматизирани тестове и интерактивна документация (Swagger UI)

Чрез командите по-долу се пускат автоматизираните тестове (pytest) от корена на проекта, след като си активирал виртуалната среда и си инсталирал зависимостите.

```
cd gp-appointments
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -q
```

Също активирайте Swagger UI, чрез което може да разцъкате интерактивна документация с извикване на примерни заявки, за да тествате функционалността на APIa.

```
uvicorn app.main:app --reload
# след това в браузър: http://127.0.0.1:8000/docs
```

2.2. Авторизация

Само следните endpointи са публични без Bearer token:

- **POST /auth/register-doctor**

```
DoctorRegisterRequest ^ Collapse all object
    name* string [1, 200] characters
    email* string email
    address* string [1, 500] characters
    password* string ≥ 6 characters
    schedule > Expand all object
    Examples > Expand all array
```

```
DoctorResponse ^ Collapse all object
    id* integer
    name* string
    email* string email
    address* string
    schedule* > Expand all object
    Examples > Expand all array
```

- POST /auth/register-patient,

```
PatientRegisterRequest ^ Collapse all object
  name* string [1, 200] characters
  email* string email
  phone* > Expand all string [3, 50] characters matches ^\+?[0-9][0-9 -]{2,49}$
  password* string ≥ 6 characters
  doctor_id* integer > 0
  Examples > Expand all array
```

```
PatientResponse ^ Collapse all object
  id* integer
  name* string
  email* string email
  phone* string
  doctor_id* integer
  Examples > Expand all array
```

- POST /auth/login

```
LoginRequest ^ Collapse all object
  email* string email
  password* string ≥ 6 characters
```

```
TokenResponse ^ Collapse all object
  access_token* string
  token_type > Expand all string
```

- GET /health (чрез което се проверява дали респондва сървъра) – без параметри

За останалите маршрути се изисква header Authorization: Bearer <access_token>, като token идва от login в полето access_token.

2.3. Формат на грешките

Обикновено тялото е JSON с ключ detail и низ на английски (както в кода). При error code 422 detail е масив от обекти с това къде се намира всеки от проблемите и съобщение за всяко проблематично поле.

2.4. Други Възможни HTTP грешки

- **200** - успех.
- **400** - нарушено бизнес-правило или логически несъвместими данни.
- **401** при липсващ, невалиден или изтекъл JWT
- **403** при валиден token, но неподходяща роля за пътя.
- **404** - липсващ ресурс
- **409** - конфликт: дублиран имейл, overlap на посещение, дублирана постоянна промяна на график за същата дата.
- **422** - невалидна структура/тип; пример: непознат ден в график, лош HH:MM, липсващо поле и т.н.

2.5. Дата, час, график

- Входът може да е ISO 8601 с или без зона; сървърът нормализира към UTC и пази naive UTC в SQLite.
- Правилата 24 h, 12 h и работно време се смятат в UTC.
- График: обект с ключове monday…sunday (без значение регистър); стойност - масив от интервали [HH:MM, HH:MM]; в рамките на ден без застъпване, сортирани, start < end; [] = неработен ден.

Пример schedule:

```
{
  "monday": [["08:30", "12:00"], ["13:00", "18:30"]],
  "tuesday": [["08:30", "12:00"], ["13:00", "18:30"]],
  "wednesday": [["08:30", "12:00"], ["13:00", "18:30"]],
  "thursday": [["08:30", "12:00"], ["13:00", "18:30"]],
  "friday": [["08:30", "12:00"], ["13:00", "18:30"]],
  "saturday": [["09:00", "12:30"]],
  "sunday": []
}
```

2.6. Лекар - /doctor (само роля лекар)

POST /doctor/schedule/temporary - Най-много една временна промяна на лекар; нова заменя старата. 200 message, details. 400 Invalid interval.

```
TemporarySchedule ChangeRequest ^ Collapse all object
  start_at* string date-time
  end_at* string date-time
  schedule* > Expand all object
```

```
MessageResponse ^ Collapse all object
  message* string
  details > Expand all (object | null)
```

POST /doctor/schedule/permanent - effective_from_date (YYYY-MM-DD), schedule. Поне 7 дни напред; 409 при вече съществуваща промяна за същата дата.

```
PermanentScheduleChangeRequest ^ Collapse all object
  effective_from_date* string date
  schedule* > Expand all object
```

```
MessageResponse ^ Collapse all object
  message* string
  details > Expand all (object | null)
```

2.7. Посещения

POST /appointments (пациент) - Лекарят трябва да е личният лекар; ≥24 h напред; един календарен ден; в ефективно работно време; без overlap с друго активно посещение на същия лекар. Възможни са 400/404/409/403 според случая.

```
AppointmentCreateRequest ^   Collapse all   object
    doctor_id*  integer  > 0
    start_at*  string  date-time
    end_at*  string  date-time
```

```
AppointmentResponse ^   Collapse all   object
    id*  integer
    doctor_id*  integer
    patient_id*  integer
    start_at*  string  date-time
    end_at*  string  date-time
    status* >   Expand all   string
    cancelled_by >   Expand all   (string | null)
    cancelled_at >   Expand all   (string | null)
    created_at*  string  date-time
```

POST /appointments/{id}/cancel - лекар или пациент по посещението; ≥12 h преди начало. 200 със status cancelled; 404 Appointment not found; 403 Not allowed; 400 при твърде късна отмяна.

```
appointment_id *  required
integer
```

```
AppointmentResponse ^  Collapse all  object
    id*  integer
    doctor_id*  integer
    patient_id*  integer
    start_at*  string  date-time
    end_at*  string  date-time
    status* >  Expand all  string
    cancelled_by >  Expand all  (string | null)
    cancelled_at >  Expand all  (string | null)
    created_at*  string  date-time
```

GET /appointments/me - масив от посещения за текущия потребител (лекар или пациент), подредени по начало. 401 при липса на валиден token.

No parameters

```
Response My Appointments Appointments Me Get ^  Collapse all  array<object>
    Items ^  Collapse all  object
        id*  integer
        doctor_id*  integer
        patient_id*  integer
        start_at*  string  date-time
        end_at*  string  date-time
        status* >  Expand all  string
        cancelled_by >  Expand all  (string | null)
        cancelled_at >  Expand all  (string | null)
        created_at*  string  date-time
        Examples >  Expand all  array
```

2.8. GET /health

Без авторизация; 200 и {"status": "ok"}.

3. Самоанализ на работата

Спазени са принципите за чист и четим код и принципи като SOLID. Описани са съзнателни компромиси и граници на дизайна.

3.1. SOLID

S (Single Responsibility)

Routers (app/routers/*) съдържат само HTTP пътища, dependency injection и мапване към response модели.

Services (app/services/*) съдържат бизнес логика и правила (24h, 12h, overlap, личен лекар, ефективен график).

Repositories (app/repositories/*) съдържат само операции към базата (SELECT/INSERT/UPDATE/DELETE).

deps.py е отделен модул само за auth dependencies (JWT, текущ потребител, role guards).

Грешките са централизирани: services хвърлят DomainError, а HTTP слоят ги превръща в статус + JSON.

O (Open/Closed)

Нови правила за посещения могат да се добавят в service слоя (например още ограничения при запис) без промяна на routers.

Нови заявки към базата се добавят като нови repository функции/методи без да се променя HTTP договорът.

Добавяне на нов тип домейн грешка става с нов подклас на DomainError, без да се променят places които я хвърлят.

L (Liskov Substitution)

Всички домейн грешки са заменими като DomainError и се обработват еднакво от централния handler.

Ролите и статусите са enums, което прави поведението предвидимо и без “изненадващи” подтипове.

Repository имплементациите могат да заменят интерфейсите (Protocol) без services да се променят.

I (Interface Segregation)

Repository интерфейсите са разделени по отговорност: users, profiles, appointments, schedule_changes (няма един “гигантски” интерфейс).

Services зависят само от това, което им трябва (например appointments service не изисква всички операции за всички таблици).

D (Dependency Inversion)

Services зависят от repository абстракции (Protocol в app/interfaces/repositories.py), а конкретната SQLAlchemy имплементация е отделно (app/repositories/sqlalchemy_impl.py).

Домейн грешките са HTTP-агностични; HTTP статусите се определят в HTTP слоя чрез mapper (app/http_error_mapping.py), а не в домейна.

Session се подава отвън (инжектира се), вместо да се създава вътре в services.

3.2. Съзнателни компромиси

- **Работно време е в JSON колона** - Работното време на лекаря (и промените по него) се пази като сериализиран JSON (schedule_json), вместо като отделни таблици за дни и интервали. Това намалява броя на таблиците и миграциите, но означава, че валидацията и логиката за графика (сортиране, застъпване, ефективен график) са в приложението, а не се налагат от SQL схемата.
- **SQLite и naive UTC** - Проектът ползва SQLite за простота на настройка и лесни тестове, без нужда от отделен DB сървър. Датите се нормализират към UTC на входа, но в базата се пазят като naive datetime стойности, което е практично за SQLite, но изисква дисциплина да се третират винаги като UTC.
- **Общи DTO за doctor/patient/appointment** - Service слоят връща малък набор общи DTO структури (DoctorDTO, PatientDTO, AppointmentDTO), вместо отделен DTO за всеки use case. Това намалява броя типове и файлове, но понякога води до това една структура да съдържа полета, които са релевантни само за част от сценариите (вместо строго “tailored” модели).
- **Dependencies (deps.py) връщат HTTPException директно** - В deps.py грешките за автентикация и права (липсващ/невалиден token, грешна роля) се връщат директно като HTTPException, а не като домейн изключения. Това е прагматично, защото dependencies са част от HTTP

слоя и така се използва стандартното поведение на FastAPI за Bearer auth, без допълнителен mapper слой.

- **Service функциите приемат SQLAlchemy Session, инжектиран от HTTP слоя**. Това е прагматичен избор за простота и ясни транзакции. При по-голяма система транзакциите биха се управлявали от отделен Unit of Work слой, но това не е необходимо за настоящия обем

3.3. Тестове

Тестовете покриват основните сценарии и изисквания чрез реални HTTP заявки към приложението, допълнени с няколко проверки на ниво service. Има основен „happy path“ сценарий, който преминава през регистрация на лекар и пациент, вход в системата, създаване на посещение и проверка, че „моите посещения“ връща коректни данни. Отделно са тествани ключовите правила — отказ при запис по-малко от 24 часа предварително, предотвратяване на застъпващи се посещения за един лекар, както и логиката за отмяна спрямо 12-часовия прозорец. Покрити са и проверки за валидност на графика (невалидни дни и припокриващи се интервали) и уникалността на имейлите. За сигурността се валидира поведението при липсващ или невалиден token и съответните HTTP кодове, а допълнителен unit тест за auth service гарантира, че невалидни креденшъли се отхвърлят.